

Gestión del Ciclo de Vida de un Modelo de Machine Learning con MLflow 3

Asignatura	MLOps
Actividad	LAB4
Autor	Carlos Molina Martínez
Fecha	19 de abril de 2026

Introducción

En esta práctica se ha implementado el ciclo de vida completo de un modelo de Machine Learning aplicado al diagnóstico de diabetes, utilizando el dataset PIMA Indians Diabetes. El flujo de trabajo comprende tres etapas principales:

1. **Experimentación masiva** mediante Grid Search con seguimiento automático en MLflow (30 combinaciones de hiperparámetros).
2. **Registro del modelo ganador** en el Model Registry de MLflow, entrenado sobre el 90% de los datos disponibles.
3. **Evaluación final** sobre el 10% de datos de test —nunca vistos por el modelo— a través de una API REST servida por MLflow.

La herramienta central de esta práctica es **MLflow 3**, que actúa como sistema de trazabilidad, almacén de artefactos y plataforma de despliegue, permitiendo reproducir cualquier experimento y auditar la decisión de selección del modelo final.

1. Tabla de Parámetros y Métricas del Modelo Registrado

1.1. Configuración del Grid Search

El experimento `diabetes_grid_search` exploró todas las combinaciones de la siguiente rejilla de hiperparámetros sobre un modelo de Regresión Logística:

Hiperparámetro	Valores explorados
<code>C</code> (regularización inversa)	0.01, 0.1, 1, 10, 100
<code>solver</code> (optimizador)	<code>liblinear</code> , <code>lbfgs</code>
<code>max_iter</code> (iteraciones máx.)	100, 500, 1000

Total de runs registrados en MLflow: $5 \times 2 \times 3 = 30$ experimentos

Las métricas evaluadas en el conjunto de **validación (≈20% de los datos de entrenamiento)** fueron: Accuracy, Precision, Recall y F1-Score.

1.2. Modelo Ganador Registrado

El modelo seleccionado y registrado en el experimento `diabetes_modelo_final` bajo el nombre `Diabetes_Modelo_Final` fue:

Parámetro	Valor
<code>C</code>	0.1
<code>solver</code>	lbfgs
<code>max_iter</code>	1000
Clase del modelo	<code>LogisticRegression</code> (scikit-learn)
Nombre en Model Registry	<code>Diabetes_Modelo_Final</code>
Run name	<code>Modelo_Ganador_C01_lbfgs</code>

Métricas obtenidas sobre el conjunto de validación

Métrica	Valor
Accuracy (Exactitud)	~0.7532
Precision (Precisión)	~0.6800
Recall (Sensibilidad)	~0.6400
F1-Score	~0.6596

Nota: las métricas de validación son estimadas a partir del historial del Grid Search; las métricas definitivas e independientes se obtienen en la Sección 3 sobre el conjunto de test.

1.3. Justificación de la Selección del Modelo — Criterio Médico

La selección del modelo ganador no se realizó únicamente en base a la métrica de **Accuracy**, sino priorizando el **Recall (Sensibilidad)**, y esta decisión responde a criterios de carácter clínico y médico-preventivo.

El problema que abordamos —el diagnóstico de diabetes— es un caso paradigmático en el que el **coste de un error no es simétrico**. Existen dos tipos de errores posibles:

- **Falso Positivo (FP):** el modelo predice que un paciente tiene diabetes, pero en realidad no la tiene. Este error es asumible porque el paciente será derivado a pruebas diagnósticas adicionales que confirmarán o descartarán el diagnóstico. Supone un coste económico moderado y cierta ansiedad para el paciente, pero no compromete su salud a largo plazo.
- **Falso Negativo (FN):** el modelo predice que un paciente está sano, pero en realidad padece diabetes. Este error es **clínicamente inaceptable** en un contexto de screening o medicina preventiva: el paciente es enviado a casa sin tratamiento, sin seguimiento y sin diagnóstico. La diabetes no tratada puede derivar en complicaciones graves como neuropatía periférica, retinopatía, insuficiencia renal o enfermedad cardiovascular.

Por tanto, el criterio de selección del modelo ganador fue maximizar el **Recall**, minimizando así los Falsos Negativos. Entre los modelos del Grid Search con Recall más elevado, se eligió aquel que ofrecía el mejor equilibrio con el F1-Score, garantizando que la Precision no cayera a niveles clínicamente irresponsables. El modelo con `C=0.1`, `solver='lbfgs'`, `max_iter=1000` ofreció este equilibrio óptimo: una regularización moderada (`C=0.1`) evita el sobreajuste, el optimizador `lbfgs` converge de forma robusta en datasets multidimensionales, y las 1000 iteraciones aseguran convergencia completa.

En medicina preventiva, **es preferible tratar a un paciente sano erróneamente que ignorar a un paciente enfermo.**

2. Script de Obtención de Métricas a través de la API

2.1. Descripción del Script

El script `5_evaluacion_test_api.py` implementa la evaluación final e independiente del modelo desplegado. Carga el dataset original completo y recrea exactamente el mismo 10% de datos de test usando `random_state=42`, garantizando que son los mismos registros que el modelo nunca vio durante el entrenamiento ni la validación. Serializa este subconjunto en formato `dataframe_split` y lo envía mediante una petición HTTP POST al endpoint `/invocations` de la API REST de MLflow (puerto 1234), recibiendo las predicciones del modelo en producción y calculando sobre ellas las cuatro métricas de evaluación definitivas.

2.2. Código Completo

```
import requests
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

# 1. Recreamos el dataset de Test usando la misma semilla (random_state=42)
df = pd.read_csv("diabetes.csv")
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Al usar random_state=42, separamos EXACTAMENTE el mismo 10% que no vio el modelo
al entrenar
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.10,
random_state=42)

# 2. Preparamos el payload en formato split para la API
payload = {
    "dataframe_split": X_test.to_dict(orient="split")
}

# 3. Hacemos la petición POST a la API (que tienes corriendo en el puerto 1234)
url = "http://localhost:1234/invocations"
print("Enviando el dataset de test (10%) a la API para evaluación final...")

try:
    response = requests.post(url, json=payload)
```

```
if response.status_code == 200:
    print("¡Predicciones recibidas con éxito!\n")

    # Extraemos las predicciones del JSON devuelto por la API
    y_pred = response.json()["predictions"]

    # 4. Calculamos las métricas finales (Métricas Reales sobre Test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    print("=== MÉTRICAS REALES SOBRE EL DATASET DE TEST ===")
    print(f"Accuracy (Exactitud) : {acc:.4f}")
    print(f"Precision (Precisión) : {prec:.4f}")
    print(f"Recall (Sensibilidad): {rec:.4f}")
    print(f"F1-Score (Equilibrio) : {f1:.4f}")
    print("=====")
    print("\n¡Práctica terminada a nivel de código!")

else:
    print(f"Error {response.status_code}: {response.text}")

except Exception as e:
    print(f"Error al conectar con la API: {e}. ¿Seguro que la dejaste encendida en la otra terminal?")
```

3. Conclusiones

3.1. Métricas Finales sobre el Dataset de Test

Tras ejecutar el script de evaluación contra la API con el modelo `Diabetes_Modelo_Final` desplegado, se obtuvieron los siguientes resultados sobre el **10% de datos de test completamente independientes**:

Métrica	Valor
Accuracy (Exactitud)	0.7013
Precision (Precisión)	0.5625
Recall (Sensibilidad)	0.6667
F1-Score	0.6102

3.2. Análisis de Generalización y Sobreajuste

Los resultados obtenidos sobre el conjunto de test revelan un comportamiento **consistente y sin indicios graves de sobreajuste**. El modelo fue entrenado con el 90% de los datos (combinando train y validación) antes de ser registrado como versión final, y al enfrentarse al 10% de test —datos que nunca había procesado— mantiene un rendimiento razonable.

La caída moderada en Accuracy (del ~75% en validación al 70.13% en test) y en Precision (del ~68% al 56.25%) es esperable y normal en modelos de Machine Learning al pasar de datos de validación a datos completamente independientes. Esta variación entra dentro de los márgenes aceptables para un modelo de regresión logística sobre el dataset PIMA, que es conocido por su complejidad y ruido intrínseco.

El dato más relevante desde la perspectiva médica es el **Recall final de 0.6667**, que significa que el modelo es capaz de identificar correctamente a **2 de cada 3 pacientes diabéticos** en datos que nunca antes había visto. Este comportamiento valida la decisión de selección del modelo ganador basada en criterios clínicos: el modelo mantiene su capacidad de detección incluso ante datos completamente nuevos, lo que lo hace adecuado para su uso como herramienta de screening.

3.3. Reflexión sobre la Utilidad de MLflow

Esta práctica ha demostrado el valor de MLflow como plataforma integral para la gestión del ciclo de vida de modelos en un contexto de MLOps:

- **Trazabilidad completa:** cada uno de los 30 experimentos del Grid Search quedó registrado con sus parámetros, métricas y artefactos, permitiendo auditar en cualquier momento qué combinación de hiperparámetros se evaluó y con qué resultado.
- **Reproducibilidad:** gracias al uso de `random_state=42` en la división de datos y al registro de parámetros exactos en MLflow, cualquier experimento puede ser reproducido de forma idéntica. La semilla fija garantiza que el 10% de test es siempre el mismo conjunto de registros.
- **Model Registry:** el componente de registro de modelos de MLflow actúa como un repositorio centralizado y versionado, separando claramente los modelos experimentales de los modelos aprobados para producción. Esto facilita la gobernanza y el control de versiones en entornos de equipo.
- **Despliegue integrado:** la capacidad de MLflow para servir modelos como API REST (`mlflow models serve`) eliminó la necesidad de escribir código de inferencia adicional. El mismo modelo registrado en el tracking server se puede desplegar con un único comando y evaluar a través de HTTP.
- **Automatización de decisiones de negocio:** la combinación de tracking + registry + serving permite institucionalizar las decisiones de selección de modelos con criterios explícitos y auditables. En un entorno clínico real, esto es esencial para cumplir con requisitos regulatorios y de transparencia algorítmica.

En definitiva, MLflow demostró ser una herramienta fundamental para profesionalizar el flujo de trabajo de Machine Learning, reduciendo la brecha entre la experimentación en local y el despliegue de modelos en producción de forma controlada y trazable.

Informe generado como entregable de la asignatura MLOps — Universidad Loyola